

Group Anagram Given an array of strings `strs`, group the anagrams together.
Return the answer in any order.

`strs = ["eat", "tea", "tan", "ate", "nat", "bat"]`

o/p: `[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]`

`strs = ["a"]`

o/p: `[["a"]]`

Intuition Same character and frequency
Order doesn't matter

eat	$\xrightarrow{\text{sort}}$	aet ✓
tea	$\xrightarrow{\text{sort}}$	aet ✓
ate	$\xrightarrow{\text{sort}}$	aet ✓
tat	$\xrightarrow{\text{sort}}$	att ✗

Brute force Sort each string $\longrightarrow O(k \log k)$
Compare each pair $\longrightarrow O(n^2)$

TC: $O(n^2 * k \log k)$

Optimized Sorted version $\longrightarrow$ key
list of original string $\longrightarrow$ values

strs = ["eat", "tea", "tan", "ate", "nat", "bat"]

o/p: [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]

eat → aet

tea → aet

tan → atn

ate → aet

"nat" → atn

bat → abt

"aet": ["eat", "tea",
"ate"]

"atn": ["tan",
"nat"]

"abt": ["bat"]

map

map.values → AL

[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]

Group Anagrams(strs) {

map // key: string, value: list of strings

for each s in strs {

ch = s.toCharArray() // convert s to character array

Arrays.sort(ch)

key = new String(ch)

if (! map.containsKey(key))

map.put(key, new ArrayList())

map.get(key).add(s)

return new ArrayList(map.values())

TC: $O(n \cdot (k \log k))$

↑
each string

↑
sort

SC: $O(n * k)$ → avg length of string

↑
no. of string